Quality-Control Pipeline

A mini agent that reads inspection reports, flags defect breaches, and measures how well it does its own job

Alex Huou

June 18, 2026

Executive summary. This tool reads manufacturing inspection reports, uses a language model to pull defect counts out of free-text inspector notes, and emails operations when a facility exceeds its quality limits. On the 12 sample reports it read 24 of 24 defect counts correctly (100%), correctly flagged the 1 facility that breached limits with 0 false alarms, and ran in milliseconds at zero cost. Crucially, the system *measures its own reliability* in plain terms a manager can act on — how accurately it reads, how confident it is, when it asks a human to double-check, and what it would cost at scale — so the people relying on it can see exactly how far to trust it.

One report — NA-CHI-021 — exercises everything this brief asks for; §4 walks it end to end. Where each requirement is demonstrated:

What you asked for	Where it's shown
Explore & understand unfamiliar data	§1 — the report schema and the free-text extraction problem
Data extraction with modern techniques	§1–§2, §5 — LLM structured-output extraction, robust to unseen phrasing
Clean, readable, testable code	§6 + Appendix B — pluggable modules, a pure rules engine, 86 tests
Explainability to non-technical audiences	§3 — the tool-quality metrics table
Logical problem-solving	§4 — the "13 hundred" case study; §5 — adversarial hardening

1 The problem, and why a language model

Each inspection report is a JSON file with hard production numbers plus two *free-text* notes written by a human auditor — one for major (critical) defects, one for minor (cosmetic) defects. The task is to alert stakeholders when defects exceed the limits:

$$\text{major} > 1\% \quad \bullet \quad \text{minor} > 3\% \quad \bullet \quad (\text{major} + \text{minor}) > 3.5\% \quad \text{of total production.}$$

The numbers are trivial to threshold; the hard part is *reading the counts out of prose that varies every time.* Across the sample alone the auditors wrote "`31 units`", "`a dozen`" (12), "`half a dozen`" (6), "`a hundred`" (100), "`a single unit`" (1), "`None`" (0), and the decisive "`13 hundred units`" (1300). A regular expression that grabs the first integer reads "13 hundred" as **13** — and that one misread is the difference between catching and missing the only true breach in the dataset. This is exactly why the brief calls for an LLM: the reading problem is open-ended natural language, not a fixed pattern.

2 How it works

The pipeline is five small, independently testable stages:

1. **Load** every report and pull out the production total and the two notes (`loader.py`), degrading gracefully — a malformed file becomes a recorded coverage gap, never a crash.
2. **Extract** the major/minor counts from the notes behind a pluggable `Extractor` interface. In **live** mode an Anthropic model returns a *structured* result (forced JSON schema); in **mock** mode a deterministic, offline parser stands in so the whole system runs with no API key at $0.

3. **Cross-check** every reading against an independent, deliberately naive rule-based reader. Where they disagree, the report is flagged for a human.
4. **Decide** via the pure threshold engine (`rules.py`; strict $>$ at every boundary).
5. **Alert** stakeholders with a plain-language email (`alerting.py`, stubbed), and **measure** the tool's own quality into `results.json`.

The mock extractor is *not* a lookup table of the sample answers — it runs the same robust number-reading logic (`number_words.py`) the live model is prompted to emulate, so it generalises to unseen wording. That is what lets us measure the tool honestly offline and ship something reproducible.

3 Measuring the tool (the part that matters)

The metrics below grade *the agent*, not the factory. They are deliberately not defect rates — they answer "how much can we trust what this tool tells us?". Every number is generated from a real run (`results.json`); none is hand-typed.

Question	Result	What it means for a non-technical reader
Did it read the counts right?	100%	Read 24/24 defect counts correctly; average error 0.0 units.
Does the LLM earn its keep?	+12.5%	A rules-only reader gets 87.5% and *misses the breach* at NA-CHI-021.
Did it catch the breaches?	100% recall	Flagged 1/1 real breach(es), 0 false alarms (precision 100%).
How sure was it?	96.5%	Mean confidence across all readings; low-confidence reads are auto-flagged.
How often ask a human?	25%	3 of 12 reports flagged for a quick human check.
How fast?	0.01 ms	Per report in offline mock; live throughput is set by the model call ($\sim$1–2 s each, run concurrently).
What does it cost?	$0	offline mock — no API calls, $0.
Did it read every file?	100%	12/12 report files read without error.

Why a cross-check, and why "ask a human". The tool runs a second, intentionally simple reader alongside the main one. The two agree on 87.5% of readings; the 3 disagreements (EU-PL-005, NA-AUS-009, NA-CHI-021) are surfaced for review. This is conservative by design: in a safety setting it is far better to ask an operator to glance at a report than to silently trust an automated read — and, satisfyingly, the one true breach is among the flagged reports, so a human is asked to confirm it.

4 Case study: "13 hundred"

Report `NA-CHI-021` produced 6,100 units; its minor-defect note reads *"13 hundred units exhibited slight connector misalignment."* The tool reads **1,300** (21.3% of production), which trips both the minor ($>$ 3%) and overall ($>$ 3.5%) rules, and emails operations. The naive cross-check reads **13** (0.2%) and would have stayed silent — a missed breach. The pipeline (i) gets it right, (ii) shows *why* (it quotes the snippet "13 hundred" as evidence in the alert), and (iii) flags the report for human confirmation because its two readers disagreed. One case, all three behaviours the brief is really asking for.

5 Robustness, and not overfitting

The brief warns that real notes will be worded differently from the sample, so the system is built to generalise and was then *adversarially stress-tested*:

- The number reader composes compound numerals left-to-right, so unseen phrasings read correctly: "sixty five" → 65, "two hundred and fifty" → 250, "twenty-five hundred" → 2500, "a couple dozen" → 24, "1.5k" → 1500. Vague quantifiers ("a few", "several") are read but kept at low confidence so they route to a human rather than passing as fact.
- A panel of independent reviewers tried to break each component (the parser, the rules, the metrics, the offline guarantee, the tests). They confirmed the core contract and surfaced real gaps — most importantly that an earlier version misread spelled-out compounds like "sixty five" as 5 at high confidence (a silent under-count), and that a failed read on a breach was being scored as a clean run. Both are fixed and now covered by tests.

6 Engineering notes

- **Modular & testable.** Pure threshold engine (no I/O), a pluggable `Extractor` interface (mock / live / baseline), and a metrics layer with a hand-verified golden set. 86 automated tests pass, including the "13 hundred" trap, every threshold boundary, the synthetic major-only / overall-only cases, an offline "no API key" proof, and a failed-read-on-the-breach safety test.
- **Live integration.** The live extractor uses Anthropic structured outputs (a forced schema), reads its key from the environment, and reports token usage so the tool can show an illustrative cost-per-report. It is drop-in: the mock and live paths share one interface, and the live path is unit-tested with a fake client.
- **Runs anywhere, $0.** `python -m qc_pipeline --mode mock` needs only `pydantic`; the Anthropic SDK is imported lazily and only for live mode.
- **Optional operator console.** A small FastAPI dashboard (`webapp/`) wraps the pipeline for non-technical users — mock/live runs, an e-mail outbox, a raw-report viewer, and a content-hash cache that avoids re-spending tokens on unchanged reports (§7). The CLI and test suite are the deliverable; the console is additive.

7 The operator console (web app)

The deliverable is the CLI and test suite; layered on top is a small FastAPI dashboard (`webapp/`, also deployed live[1]) that lets a non-technical operator *watch* the pipeline work: run it in mock or live mode, see every tool-quality metric update, drag in new reports, and — the two views below — track what the tool would send and inspect exactly what tripped an alert.

E-mail outbox. The stubbed `send_email()` (`alerting.py`) surfaces as a tracker (Fig. 1): each breach becomes a SENT alert and each review-flagged report a DRAFT, every one openable to read the exact rendered message (Fig. 2a). The body is an editable template, so an operator can tune the wording without touching code. Nothing is actually e-mailed — the stub logs the message — and the panel says so explicitly, so the demo can never be mistaken for live sending.

Report viewer. Clicking an alert (or any ledger row) opens the underlying inspection JSON with the decision-relevant fields highlighted (Fig. 2b): the production total and the two free-text notes. For `NA-CHI-021` the highlighted minor note — *"13 hundred units…"* — is the whole story: the tool read 1,300, the naive cross-check read 13, and the panel surfaces both so a human can confirm in seconds. A file that fails to parse opens the same way with the offending field flagged, making the "coverage gap, not a crash" behaviour visible rather than buried in a log.

[1] `mini-agent-qc.vercel.app` — the offline analysis runs on load at $0; live extraction is optional and runs on a budgeted demo key or your own.

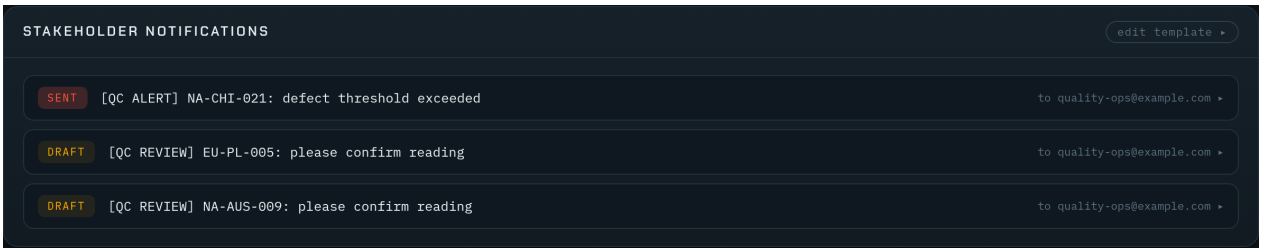
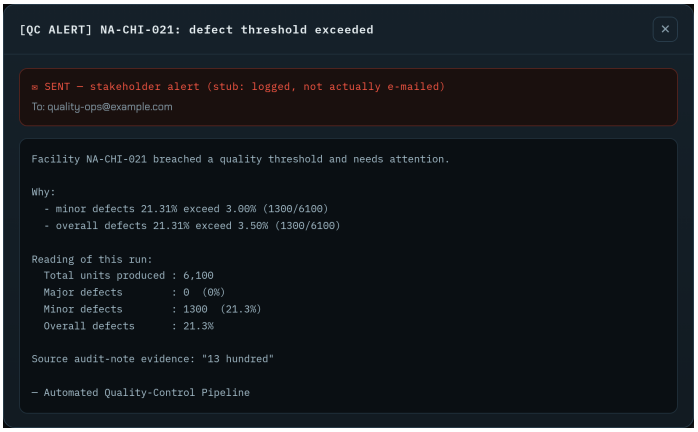


Figure 1: The outbox turns the `send_email()` stub into something a manager can see: the one breach (NA-CHI-021) is a SENT alert; the two reports the readers disagreed on are DRAFT review requests. Nothing leaves the machine.



(a) The rendered alert e-mail: a plain-language "why", the reading, and the quoted evidence "13 hundred". The header marks it a logged stub, not a sent message.



(b) The raw-JSON viewer with the total and the two audit notes highlighted — the minor note "13 hundred units…" is what drives the breach.

Figure 2: Click an alert to see what the tool would send (left) and what tripped it (right).

8 Limitations & future work

The offline parser is a faithful but bounded stand-in for the model: it reads the first quantity in a note (fine for single-count notes, ambiguous if a note mixes several numbers), and treats vague quantifiers as low-confidence estimates. The golden set is the 12 provided reports; on genuinely new data the tool still reports confidence, speed, coverage and alerts, but cannot score its own accuracy without labels. Natural extensions: a small human-labelled validation set refreshed periodically, per-line-item extraction when notes carry multiple counts, and trend dashboards across runs.

A Running it

```
pip install -r requirements.txt
python -m qc_pipeline --mode mock       # offline, deterministic, $0
python -m pytest -q                     # full test suite

# Live (real model extraction):
cp .env.example .env                    # set ANTHROPIC_API_KEY
python -m qc_pipeline --mode live --tier balanced   # cheap=Haiku / balanced=Sonnet / max=Opus
```

B Key source: the number reader and the rules engine

Listing 1: qc_pipeline/extraction/number_words.py — robust natural-language quantity parser

```python
"""Robust natural-language quantity parser.

This is the heart of why the task needs more than a naive regex. Audit notes
phrase counts every which way:

    "31 units"              -> 31      "13 hundred units"     -> 1300  (the trap)
    "a hundred units"       -> 100     "two hundred and fifty"-> 250
    "fourteen hundred"      -> 1400    "twenty-five hundred"  -> 2500
    "sixty five units"      -> 65      "one thousand two hundred" -> 1200
    "a dozen"               -> 12      "half a dozen"         -> 6
    "a single unit"         -> 1       "1.5k units"           -> 1500
    "a couple dozen"        -> 24 (low confidence)
    "No critical failures" -> 0        "did not find any"     -> 0

``parse_count`` returns the value, a confidence in [0, 1] reflecting how
unambiguous the phrasing was (fuzzy quantifiers stay low so they route to human
review), the matched evidence, and the method used.

The composer walks the note left-to-right and accumulates spelled-out numerals
with the standard ones/tens + hundred/thousand algorithm, so compound numbers
("sixty five", "two hundred and fifty") and multipliers ("13 hundred") are read
correctly rather than collapsing to a single token. This robust parser backs the
offline mock extractor (a deterministic stand-in for the LLM); a deliberately
naive baseline lives separately in ``baseline_extractor.py``.
"""

from __future__ import annotations

import re
from dataclasses import dataclass


@dataclass
class NumberParse:
    value: int
    confidence: float
    evidence: str
    method: str


_ONES = {
    "zero": 0, "one": 1, "two": 2, "three": 3, "four": 4, "five": 5, "six": 6,
    "seven": 7, "eight": 8, "nine": 9, "ten": 10, "eleven": 11, "twelve": 12,
    "thirteen": 13, "fourteen": 14, "fifteen": 15, "sixteen": 16,
    "seventeen": 17, "eighteen": 18, "nineteen": 19,
}
_TENS = {
    "twenty": 20, "thirty": 30, "forty": 40, "fourty": 40,  # 'fourty' = common misspelling
    "fifty": 50, "sixty": 60, "seventy": 70, "eighty": 80, "ninety": 90,
}
_MUL_SMALL = {"hundred": 100, "dozen": 12}                  # multiply the running group
_MUL_BIG = {"thousand": 1000, "million": 1_000_000, "billion": 1_000_000_000}
# Words that connect/qualify a numeral without contributing a value of their own.
_FILLER = {"a", "an", "and", "of", "the", "single", "sole", "lone"}

_FUZZY_BASE = {"couple": 2, "few": 3, "several": 4, "handful": 5, "bunch": 5, "numerous": 6}

_K_RE = re.compile(r"\b(\d+(?:\.\d+)?)\s*k\b")
_HALF_DOZEN_RE = re.compile(r"\bhalf\s+(?:a\s+)?dozen\b")
_SINGLE_RE = re.compile(r"\b(?:a|one)\s+(?:single|sole|lone)\b")
_FUZZY_RE = re.compile(
    r"\b(?:a\s+)?(couple|few|several|handful|bunch|numerous)(?:\s+of)?"
    r"(?:\s+(dozen|hundred|thousand|million))?\b"
)
_ABSENCE_RE = re.compile(r"\b(no|none|not|without|nil|never|zero)\b")
_TOKEN_RE = re.compile(r"[a-z]+|\d[\d,]*")


def _compose(text: str):
    """Accumulate the first spelled/numeric quantity in ``text``.
```

```
71
72      Returns (value, used_scale, used_digit, evidence) or None if no numeral
73      starts. Hyphens are normalised to spaces so "twenty-five" reads as 25.
74      """
75      tokens = _TOKEN_RE.findall(text.replace("-", " "))
76      result = 0          # accumulated thousands+ groups
77      current = 0         # the group currently being built
78      used = used_scale = used_digit = False
79      consumed: list[str] = []
80
81      for tok in tokens:
82          if tok[0].isdigit():
83              current += int(tok.replace(",", ""))
84              used = used_digit = True
85              consumed.append(tok)
86          elif tok in _ONES:
87              current += _ONES[tok]; used = True; consumed.append(tok)
88          elif tok in _TENS:
89              current += _TENS[tok]; used = True; consumed.append(tok)
90          elif tok in _MUL_SMALL:
91              current = (current or 1) * _MUL_SMALL[tok]
92              used = used_scale = True; consumed.append(tok)
93          elif tok in _MUL_BIG:
94              result += (current or 1) * _MUL_BIG[tok]
95              current = 0; used = used_scale = True; consumed.append(tok)
96          elif tok in _FILLER:
97              continue            # connective/qualifier: neither counts nor stops
98          else:
99              if used:
100                 break           # a real word after the numeral ends it
101             continue            # leading prose before the numeral - skip
102     if not used:
103         return None
104     return result + current, used_scale, used_digit, " ".join(consumed)
105
106
107 def parse_count(text: str) -> NumberParse:
108     """Extract a single defect count from a free-text audit note."""
109     if not text or not text.strip():
110         return NumberParse(0, 0.20, "", "empty")
111     t = text.lower()
112
113     # 1. "1.5k" / "2k".
114     if (m := _K_RE.search(t)):
115         return NumberParse(int(round(float(m.group(1)) * 1000)), 0.90, m.group(0), "k_suffix")
116
117     # 2. "half a dozen" idiom.
118     if (m := _HALF_DOZEN_RE.search(t)):
119         return NumberParse(6, 0.90, m.group(0), "dozen")
120
121     # 3. Fuzzy quantifiers (only when no explicit digit is present, so an exact
122     #    number always wins). Imprecise -> low confidence -> routed to review.
123     if not any(c.isdigit() for c in t) and (m := _FUZZY_RE.search(t)):
124         base = _FUZZY_BASE[m.group(1)]
125         scale_word = m.group(2)
126         scale = _MUL_SMALL.get(scale_word) or _MUL_BIG.get(scale_word) or 1
127         return NumberParse(base * scale, 0.50, m.group(0), "fuzzy")
128
129     # 4. Exact composition: digits, spelled numerals, multipliers, additive tails.
130     composed = _compose(t)
131     if composed is not None:
132         value, used_scale, used_digit, evidence = composed
133         if used_digit and not used_scale and len(evidence.split()) == 1:
134             conf, method = 0.99, "digit"        # a bare integer like "31"
135         elif used_scale:
136             conf, method = 0.92, "scale"        # "13 hundred", "a hundred", "two thousand"
137         else:
138             conf, method = 0.90, "word"         # "sixty five", "nineteen"
139         return NumberParse(value, conf, evidence, method)
140
141     # 5. "a single / a sole / a lone" -> 1 (idiom with no numeric token).
142     if (m := _SINGLE_RE.search(t)):
143         return NumberParse(1, 0.90, m.group(0), "word")
144
145     # 6. Explicit absence / negation -> 0.
```

```
    if (m := _ABSENCE_RE.search(t)):
        return NumberParse(0, 0.96, m.group(0), "absence")

    # 7. Nothing parseable - default to 0 but flag low confidence for review.
    return NumberParse(0, 0.25, "", "none_found")
```

Listing 2: qc_pipeline/rules.py — the pure defect-threshold engine

```
"""The defect-threshold engine - pure, deterministic, dependency-free.

An alert fires if ANY of these is exceeded (STRICT greater-than; a value sitting
exactly on the threshold does NOT alert):

    major   / total  >  1.0%
    minor   / total  >  3.0%
    (major+minor)/total  >  3.5%

This module has no knowledge of LLMs, files, or e-mail - it takes three numbers
and the thresholds and returns a decision. That keeps it trivially testable and
lets the boundary behaviour be pinned down exactly.
"""

from __future__ import annotations

from dataclasses import dataclass, field

from .config import Config


@dataclass
class AlertDecision:
    site_code: str
    total_products: int
    major: int
    minor: int
    major_pct: float
    minor_pct: float
    overall_pct: float
    alert: bool
    reasons: list[str] = field(default_factory=list)


def evaluate_thresholds(
    site_code: str,
    total_products: int,
    major: int,
    minor: int,
    *,
    major_threshold: float,
    minor_threshold: float,
    overall_threshold: float,
) -> AlertDecision:
    """Apply the three OR-ed threshold rules and explain which (if any) tripped."""
    # Guard against division by zero; a non-positive batch can't exceed any rate.
    if total_products <= 0:
        return AlertDecision(site_code, total_products, major, minor, 0.0, 0.0, 0.0, False,
                             ["non-positive total_products - no rate computable"])

    major_pct = major / total_products
    minor_pct = minor / total_products
    overall_pct = (major + minor) / total_products

    reasons: list[str] = []
    if major_pct > major_threshold:
        reasons.append(
            f"major defects {major_pct:.2%} exceed {major_threshold:.2%} ({major}/{
                total_products})"
        )
    if minor_pct > minor_threshold:
        reasons.append(
            f"minor defects {minor_pct:.2%} exceed {minor_threshold:.2%} ({minor}/{
                total_products})"
        )
    if overall_pct > overall_threshold:
        reasons.append(
```

```
            f"overall defects {overall_pct:.2%} exceed {overall_threshold:.2%} "
            f"({major + minor}/{total_products})"
        )

    return AlertDecision(
        site_code=site_code,
        total_products=total_products,
        major=major,
        minor=minor,
        major_pct=major_pct,
        minor_pct=minor_pct,
        overall_pct=overall_pct,
        alert=bool(reasons),
        reasons=reasons,
    )


def evaluate(site_code: str, total_products: int, major: int, minor: int, config: Config) -> AlertDecision:
    """Convenience wrapper that pulls thresholds from a ``Config``."""
    return evaluate_thresholds(
        site_code,
        total_products,
        major,
        minor,
        major_threshold=config.major_threshold,
        minor_threshold=config.minor_threshold,
        overall_threshold=config.overall_threshold,
    )
```